

實驗室 3：從原型到正式環境 - 將 ADK 代理部署至搭載 GPU 的 Cloud Run

來源：<https://codelabs.developers.google.com/codelabs/cloud-run/how-to-connect-adk-to-deployed-cloud-run-llm?hl=zh-tw>

步驟數：13

瞭解如何將已部署的 ADK 代理程式連結至已部署的 Gemma 模型，打造可調度資源的 AI 應用程式。

目錄

1. [1. 簡介](#)
2. [2. 專案設定](#)
3. [3. 開啟 Cloud Shell 編輯器](#)
4. [4. 啟用 API 並設定預設區域](#)
5. [5. 準備 Python 專案](#)
6. [6. 架構總覽](#)
7. [7. 將 Gemma 後端部署至搭載 GPU 的 Cloud Run](#)
8. [8. 導入 ADK 代理整合](#)
9. [9. 設定環境並部署代理程式](#)
10. [10. 使用 ADK 網頁介面進行測試](#)
11. [11. 導入及執行彈性測試](#)
12. [12. 觀察自動調度資源行為](#)
13. [13. 結論](#)

步驟 1 · 約 0 分鐘

1. 簡介

總覽

在本實驗室中，您將部署可供正式環境使用的 Agent Development Kit (ADK) 代理，並搭配 GPU 加速的 Gemma 後端。重點在於重要的部署模式：設定已啟用 GPU 的 Cloud Run 服務、將模型後端與 ADK 代理程式整合，以及觀察負載下的自動調度資源行為。

學習內容

在本實驗室中，您將著重於重要的正式環境部署層面：

1. 將 **Gemma** 部署至 **Cloud Run GPU** - 設定高效能的 Gemma 模型後端
2. 將 **Gemma** 部署作業與 **ADK** 代理整合 - 將代理連結至 GPU 加速模型
3. 使用 **ADK** 網頁介面測試 - 驗證對話式代理是否正常運作
4. 執行彈性測試 - 觀察 Cloud Run 執行個體如何處理負載

重點在於生產部署模式，而非廣泛的代理程式開發。

課程內容

- 將 GPU 加速的 Gemma 模型部署至 Cloud Run，以用於實際工作環境
 - 將外部模型部署項目與 ADK 代理整合
 - 設定及測試可用於正式環境的 AI 代理部署作業
 - 瞭解負載下的 Cloud Run 行為
 - 觀察多個 Cloud Run 執行個體在流量尖峰期間如何協調運作
 - 套用彈性測試來驗證效能
-

2. 專案設定

1. 如果沒有 Google 帳戶，請先[建立帳戶](#)。
 - 請改用個人帳戶，而非公司或學校帳戶。公司和學校帳戶可能設有限制，導致您無法啟用本實驗室所需的 API。
 2. 登入 [Google Cloud 控制台](#)。
 3. 在 Cloud 控制台中[啟用帳單](#)。
 - 完成本實驗室的 Cloud 資源費用應不到 \$1 美元。
 - 您可以按照本實驗室結尾的步驟刪除資源，以免產生後續費用。
 - 新使用者可獲得價值 [\\$300 美元的免費試用期](#)。
 4. [建立新專案](#)，或選擇重複使用現有專案。
 - 如果看到專案配額相關錯誤，請重複使用現有專案，或刪除現有專案來建立新專案。
-

步驟 3 · 約 0 分鐘

3. 開啟 Cloud Shell 編輯器

1. 按一下這個連結，直接前往 [Cloud Shell 編輯器](#)
2. 如果系統在今天任何時間提示您授權，請點選「授權」繼續操作。

Authorize Cloud Shell

gcloud is requesting your credentials to make a GCP API call.

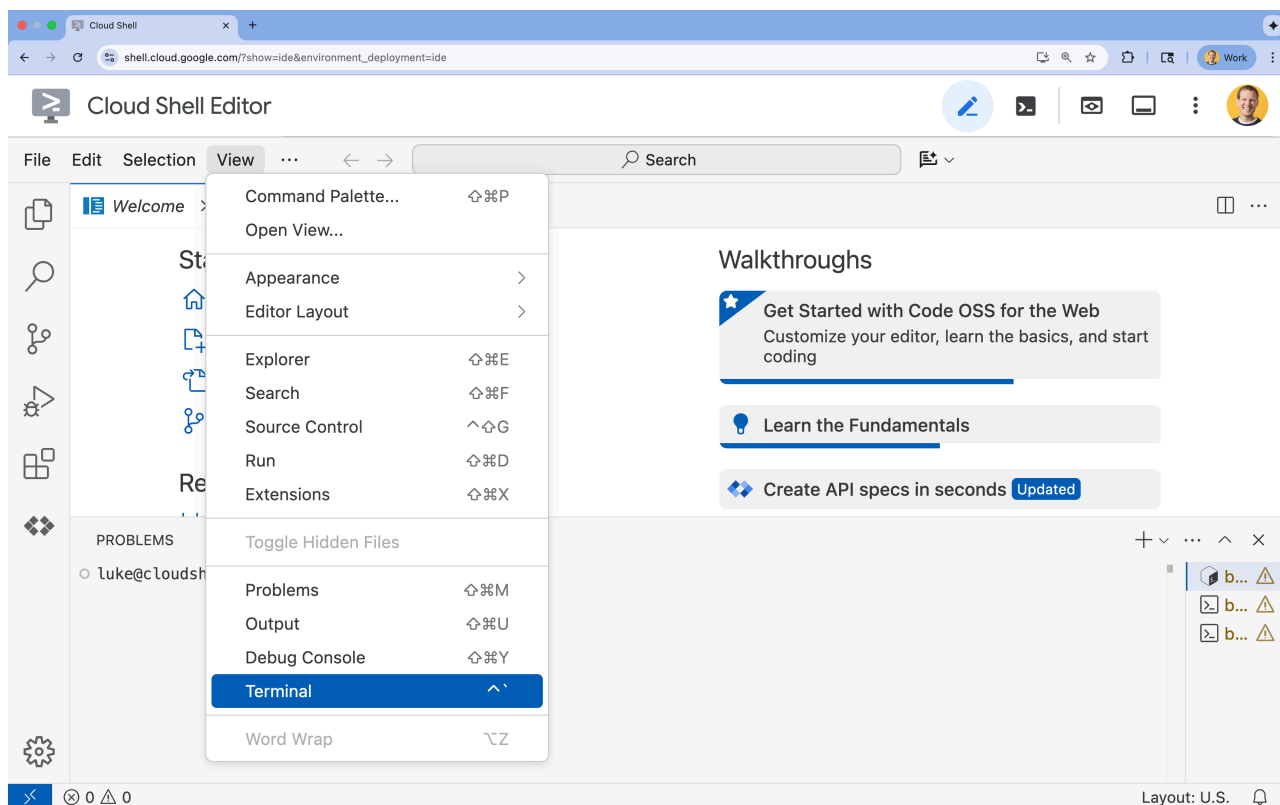
Click to authorize this and future calls that require your credentials.

REJECT

AUTHORIZE

3. 如果畫面底部未顯示終端機，請開啟終端機：

- 按一下「查看」
- 按一下「終端機」



4. 在終端機中，使用下列指令設定專案：

- 格式：

```
gcloud config set project [PROJECT_ID]
```

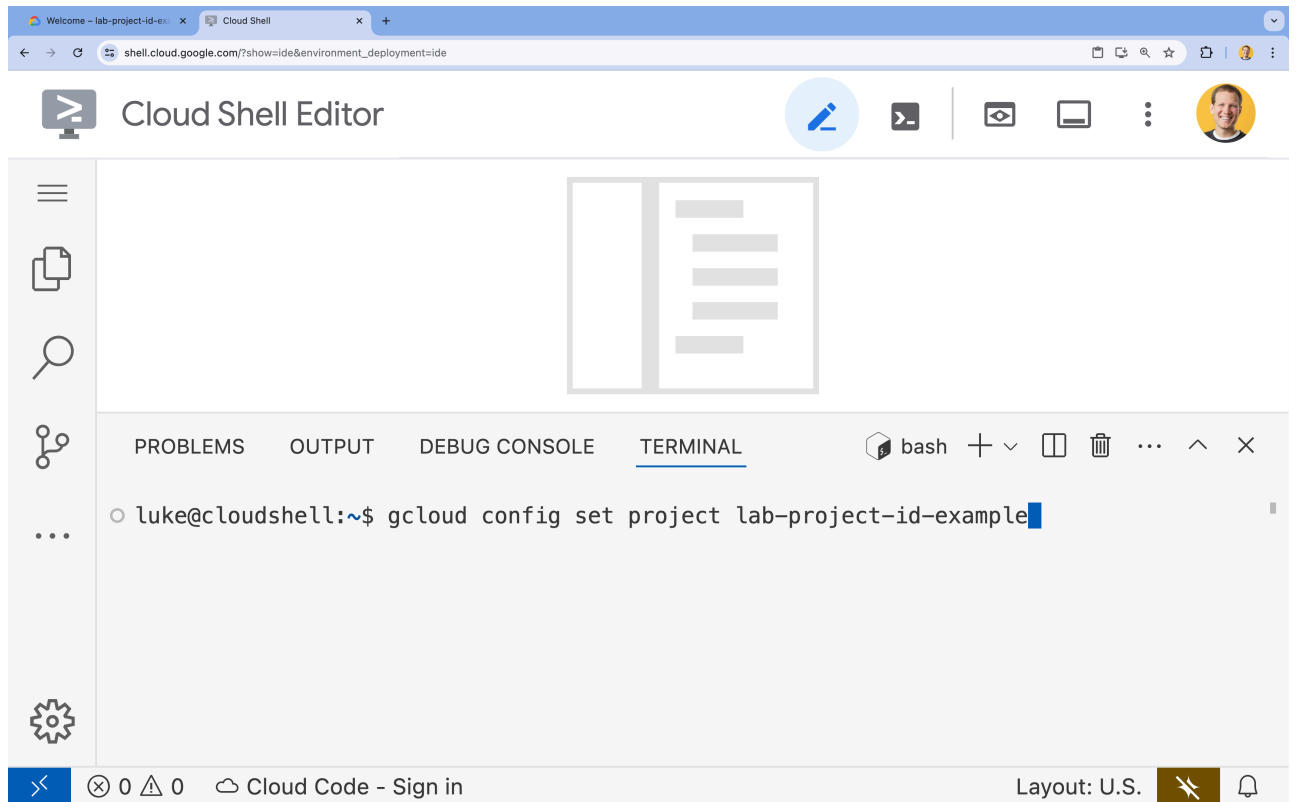
- 範例：

```
gcloud config set project lab-project-id-example
```

◦ 如果忘記專案 ID，請按照下列步驟操作：

- 您可以使用下列指令列出所有專案 ID：

```
gcloud projects list | awk '/PROJECT_ID/{print $2}'
```



5. 您應會看到下列訊息：

```
Updated property [core/project].
```

如果看到 `WARNING` 並收到 `Do you want to continue (Y/n)?` 提示，可能是專案 ID 輸入有誤。按下 `n` 和 `Enter`，然後再次嘗試執行 `gcloud config set project` 指令。

步驟 4 · 約 0 分鐘

4. 啟用 API 並設定預設區域

如要部署支援 GPU 的 Cloud Run 服務，請先啟用必要的 Google Cloud API，並設定專案設定。

1. 在終端機中啟用 API：

```
gcloud services enable \  
  run.googleapis.com \  
  artifactregistry.googleapis.com \  
  cloudbuild.googleapis.com \  
  aiplatform.googleapis.com
```

如果系統提示您授權，請點選「授權」繼續操作。

Authorize Cloud Shell

gcloud is requesting your credentials to make a GCP API call.

Click to authorize this and future calls that require your credentials.

REJECT

AUTHORIZE

這個指令可能需要幾分鐘才能完成，但最終應該會產生類似以下的成功訊息：

```
Operation "operations/acf.p2-73d90d00-47ee-447a-b600" finished successfully.
```

3. 設定預設 Cloud Run 地區。

```
gcloud config set run/region europe-west4
```

步驟 5 · 約 0 分鐘

5. 準備 Python 專案

讓我們設定包含 Gemma 後端和 ADK 代理服務基本架構的範例程式碼。

1. 複製範例存放區：

```
cd ~
npx -y giget@latest gh+git:GoogleCloudPlatform/devrel-demos/agents/accelerate-ai-with-
cloud-run/accelerate-ai-lab3-starter accelerate-ai-lab3-starter
cd accelerate-ai-lab3-starter
```

2. 檢查專案結構：

```
ls -R
```

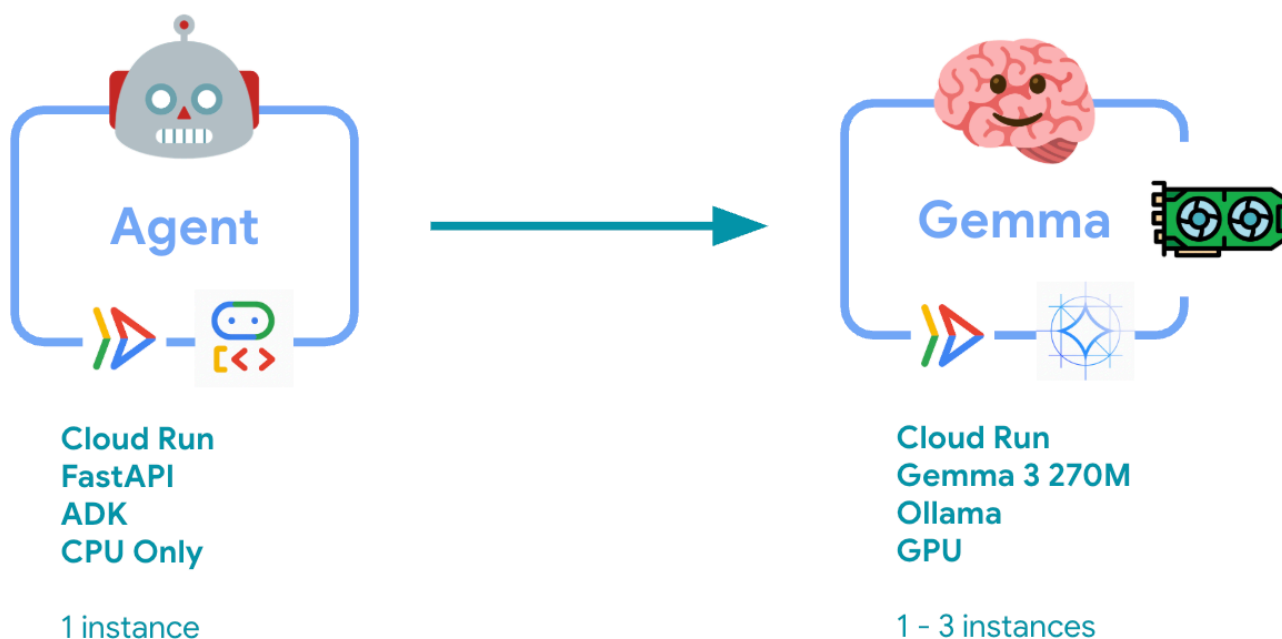
您應該會看到下列起始結構：

```
accelerate-ai-lab3-starter/
├── README.md                # Project overview
├── ollama-backend/          # Ollama backend (separate deployment)
│   └── Dockerfile           # Backend container (🚧 to implement)
└── adk-agent/               # ADK agent (separate deployment)
    ├── pyproject.toml       # Python dependencies (✅ completed)
    ├── server.py            # FastAPI server (🚧 to implement)
    ├── Dockerfile           # Container config (🚧 to implement)
    ├── elasticity_test.py   # Elasticity testing (🚧 to implement)
    └── production_agent/    # Agent implementation
        ├── __init__.py      # Package init (✅ completed)
        └── agent.py         # Agent logic (🚧 to implement)
```


步驟 6 · 約 0 分鐘

6. 架構總覽

實作之前，請先瞭解雙服務架構：



重要洞察：在彈性測試期間，您會發現這兩項服務各自處理工作負載：GPU 後端 (瓶頸服務) 使用 GPU 處理負載，而 ADK 代理程式則依賴 CPU 處理非資源密集型要求。

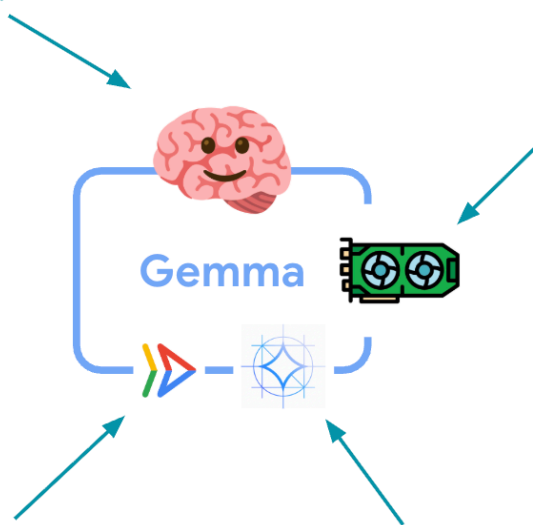
7. 將 Gemma 後端部署至搭載 GPU 的 Cloud Run

The Brain: This service will act as the brain for our application.

GPU: Since LLMs require a lot of resources to run, we are going to take advantage of Cloud Run's ability to deploy with GPU support.

Cloud Run: Our server will be deployed to Cloud Run, giving us a variety of benefits!

Gemma 3 270M: We are going to be using Gemma 3 270M as our Open Model.



首先，請部署 GPU 加速的 Gemma 模型，做為 ADK 代理程式的大腦。在需要個別微調模型或獨立調度資源的架構中，解耦並部署 LLM 可能會比較合適。

1. 前往 Ollama 後端目錄：

```
cd ollama-backend
```

2. 開啟並實作 Ollama Dockerfile：

```
cloudshell edit Dockerfile
```

將 TODO 註解替換為：

```
FROM ollama/ollama:latest

# Listen on all interfaces, port 8080
ENV OLLAMA_HOST 0.0.0.0:8080

# Store model weight files in /models
ENV OLLAMA_MODELS /models

# Reduce logging verbosity
ENV OLLAMA_DEBUG false

# Never unload model weights from the GPU
ENV OLLAMA_KEEP_ALIVE -1

# Store the model weights in the container image
ENV MODEL gemma3:270m
RUN ollama serve & sleep 5 && ollama pull $MODEL

# Start Ollama
ENTRYPOINT ["ollama", "serve"]
```

🔧 這項功能的作用：

- 以官方 Ollama 映像檔為基礎
- 將 `OLLAMA_HOST` 設為接受來自任何 IP 位址的連線
- 公開通訊埠 8080

3. 部署支援 GPU 的 Gemma 後端：

注意：這項部署作業最多可能需要 5 分鐘才能完成。這是因為系統會提取模型，例如 Gemma 3 270M。此外，佈建 GPU 的時間會比僅支援 CPU 的 Cloud Run 執行個體稍長。

```
gcloud run deploy ollama-gemma3-270m-gpu \
  --source . \
  --region europe-west4 \
  --concurrency 7 \
  --cpu 8 \
  --set-env-vars OLLAMA_NUM_PARALLEL=4 \
  --gpu 1 \
  --gpu-type nvidia-l4 \
  --max-instances 1 \
  --memory 16Gi \
  --allow-unauthenticated \
  --no-cpu-throttling \
  --no-gpu-zonal-redundancy \
  --timeout 600 \
  --labels dev-tutorial=codelab-agent-gpu
```

如果收到「Deploying from source requires an Artifact Registry Docker repository to store built containers. 系統會顯示「A repository named [cloud-run-source-deploy] in region [europe-west4] will be created." (系統將在 [europe-west4] 區域中建立名為 [cloud-run-source-deploy] 的存放區。) 訊息，請繼續操作。

⚙️ 重要設定說明：

- **GPU**：選擇 NVIDIA L4 是因為其推論工作負載的性價比極高。L4 提供 24 GB 的 GPU 記憶體，並最佳化張量運算，因此非常適合 Gemma 等 2.7 億參數模型
- **記憶體**：16 GB 系統記憶體，用於處理模型載入、CUDA 作業和 Ollama 的記憶體管理
- **CPU**：8 核心，可妥善處理 I/O 和前置處理工作
- **並行**：每個執行個體 7 個要求，可平衡處理量與 GPU 記憶體用量
- **逾時**：600 秒可容納初始模型載入和容器啟動

💰 **費用考量**：GPU 執行個體比僅含 CPU 的執行個體貴上許多 (每小時約 \$2 至 \$4 美元，僅含 CPU 的執行個體則為每小時約 \$0.10 美元)。 `--max-instances 1` 設定可避免不必要的 GPU 執行個體擴充，有助於控管費用。

4. 等待部署作業完成，並記下服務網址：

```
export OLLAMA_URL=$(gcloud run services describe ollama-gemma3-270m-gpu \
  --region=europe-west4 \
  --format='value(status.url)')

echo "🎉 Gemma backend deployed at: $OLLAMA_URL"
```

步驟 8 · 約 0 分鐘

8. 導入 ADK 代理整合

現在來建立可連線至已部署 Gemma 後端的 ADK 代理。

1. 前往 ADK 代理目錄：

```
cd ../adk-agent
```

2. 開啟並實作代理程式設定：

```
cloudshell edit production_agent/agent.py
```

將所有 TODO 註解替換為這個最簡單的實作項目：

```

import os
from pathlib import Path

from dotenv import load_dotenv
from google.adk.agents import Agent
from google.adk.models.lite_llm import LiteLlm
import google.auth

# Load environment variables
root_dir = Path(__file__).parent.parent
dotenv_path = root_dir / ".env"
load_dotenv(dotenv_path=dotenv_path)

# Configure Google Cloud
try:
    _, project_id = google.auth.default()
    os.environ.setdefault("GOOGLE_CLOUD_PROJECT", project_id)
except Exception:
    pass

os.environ.setdefault("GOOGLE_CLOUD_LOCATION", "europe-west4")

# Configure model connection
gemma_model_name = os.getenv("GEMMA_MODEL_NAME", "gemma3:270m")
api_base = os.getenv("OLLAMA_API_BASE", "localhost:10010") # Location of Ollama server

# Production Gemma Agent - GPU-accelerated conversational assistant
production_agent = Agent(
    model=LiteLlm(model=f"ollama_chat/{gemma_model_name}", api_base=api_base),
    name="production_agent",
    description="A production-ready conversational assistant powered by GPU-accelerated Gemma.",
    instruction="""You are 'Gem', a friendly, knowledgeable, and enthusiastic zoo tour guide.

    Your main goal is to make a zoo visit more fun and educational for guests by answering their questions.

    You can provide general information and interesting facts about different animal species, such as:
    - Their natural habitats and diet. 🌳🍓
    - Typical lifespan and behaviors.
    - Conservation status and unique characteristics.

    IMPORTANT: You do NOT have access to any tools. This means you cannot look up real-time, specific information about THIS zoo. You cannot provide:
    - The names or ages of specific animals currently at the zoo.
    - The exact location or enclosure for an animal.
    - The daily schedule for feedings or shows.

    Always answer based on your general knowledge about the animal kingdom. Keep your tone cheerful, engaging, and welcoming for visitors of all ages. 🧙✨""",
    tools=[], # Gemma focuses on conversational capabilities

```

```
)  
  
# Set as root agent  
root_agent = production_agent
```

🔧 這項功能的作用：

- 透過 LiteLm 連線至已部署的 Gemma 後端
- 建立簡單的對話式代理
- 設定 Google Cloud 整合

3. 開啟並實作 FastAPI 伺服器：

```
cloudshell edit server.py
```

將所有 TODO 註解替換為：

```

import os
from dotenv import load_dotenv
from fastapi import FastAPI
from google.adk.cli.fast_api import get_fast_api_app

# Load environment variables
load_dotenv()

AGENT_DIR = os.path.dirname(os.path.abspath(__file__))
app_args = {"agents_dir": AGENT_DIR, "web": True}

# Create FastAPI app with ADK integration
app: FastAPI = get_fast_api_app(**app_args)

# Update app metadata
app.title = "Production ADK Agent - Lab 3"
app.description = "Gemma agent with GPU-accelerated backend"
app.version = "1.0.0"

@app.get("/health")
def health_check():
    return {"status": "healthy", "service": "production-adk-agent"}

@app.get("/")
def root():
    return {
        "service": "Production ADK Agent - Lab 3",
        "description": "GPU-accelerated Gemma agent",
        "docs": "/docs",
        "health": "/health"
    }

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8080, log_level="info")

```

這項功能的作用：

- 建立整合 ADK 的 FastAPI 伺服器
- 啟用網頁介面以進行測試
- 提供健康狀態檢查端點

4. 開啟並實作 Dockerfile：

```
cloudshell edit Dockerfile
```

將所有 TODO 註解替換為：


```
FROM python:3.13-slim

# Copy uv from the official image
COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv

# Install system dependencies
RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*

# Set working directory
WORKDIR /app

# Copy all files
COPY . .

# Install Python dependencies
RUN uv sync

# Expose port
EXPOSE 8080

# Run the application
CMD ["uv", "run", "uvicorn", "server:app", "--host", "0.0.0.0", "--port", "8080"]
```

技術選擇說明：

- **uv**：新式 Python 套件管理員，速度比 pip 快 10 到 100 倍。這項工具使用全域快取和並行下載，大幅縮短容器建構時間
 - **Python 3.13-slim**：最新 Python 版本，系統依附元件最少，可縮減容器大小和攻擊面
 - **多級式建構**：從官方映像檔複製 uv，確保我們取得最新的最佳化二進位檔
-

9. 設定環境並部署代理程式

現在我們要設定 ADK 代理程式，連線至已部署的 Gemma 後端，並將其部署為 Cloud Run 服務。包括設定環境變數，以及使用正確的設定部署代理程式。

注意：開始本節內容前，請確認您位於 `/adk-agent` 目錄。

1. 設定環境設定：

```
cat << EOF > .env
GOOGLE_CLOUD_PROJECT=$(gcloud config get-value project)
GOOGLE_CLOUD_LOCATION=europe-west4
GEMMA_MODEL_NAME=gemma3:270m
OLLAMA_API_BASE=$OLLAMA_URL
EOF
```

瞭解 Cloud Run 中的環境變數

環境變數是鍵/值組合，可在執行階段設定應用程式。特別適合用於：

- API 端點和服務網址 (例如 Ollama 後端)
- 在不同環境 (開發、預先發布、正式版) 之間變更的設定
- 不應硬式編碼的機密資料

部署 ADK 代理：

```
export PROJECT_ID=$(gcloud config get-value project)

gcloud run deploy production-adk-agent \
  --source . \
  --region europe-west4 \
  --allow-unauthenticated \
  --memory 4Gi \
  --cpu 2 \
  --max-instances 1 \
  --concurrency 50 \
  --timeout 300 \
  --set-env-vars GOOGLE_CLOUD_PROJECT=$PROJECT_ID \
  --set-env-vars GOOGLE_CLOUD_LOCATION=europe-west4 \
  --set-env-vars GEMMA_MODEL_NAME=gemma3:270m \
  --set-env-vars OLLAMA_API_BASE=$OLLAMA_URL \
  --labels dev-tutorial=codelab-agent-gpu
```

重要設定：

- **自動調度資源：**固定為 1 個執行個體 (輕量要求處理)
- **並行：**每個執行個體 50 個要求

- 記憶體：ADK 代理程式需要 4 GB
- 環境：連線至 Gemma 後端

 **安全性注意事項：**為簡化程序，本實驗室使用 `--allow-unauthenticated`。在實際工作環境中，請使用下列方式實作適當的驗證機制：

- 使用服務帳戶進行 Cloud Run 服務對服務驗證
- 身分與存取權管理 (IAM) 政策
- 外部存取適用的 API 金鑰或 OAuth
- 考慮使用 `gcloud run services add-iam-policy-binding` 控制存取權

取得代理程式服務網址：

```
export AGENT_URL=$(gcloud run services describe production-adk-agent \
  --region=europe-west4 \
  --format='value(status.url)')

echo "🚀 ADK Agent deployed at: $AGENT_URL"
```

✅ 環境變數最佳做法，依據 [Cloud Run 環境變數說明文件](#)：

1. 避免使用保留變數：請勿設定 `PORT` (如需變更環境變數，請改用 `-port` 標記)，或以 `X_GOOGLE_` 開頭的變數
2. 使用描述性名稱：為變數加上前置字元，避免發生衝突 (例如 `GEMMA_MODEL_NAME` 而非 `MODEL`)
3. 逸出逗號：如果值含有逗號，請使用其他分隔符號：`--set-env-vars "^^KEY1=value1,value2@KEY2=..."`
4. 更新與取代：使用 `--update-env-vars` 新增/變更特定變數，而不影響其他變數

如何在 Cloud Run 中設定變數：

- 從檔案：`gcloud run deploy SERVICE_NAME --env-vars-file .env --labels dev-tutorial codelab-adk` (從檔案載入多個變數)
 - 多個標記：針對無法以半形逗號分隔的複雜值，請重複 `--set-env-vars`
-

10. 使用 ADK 網頁介面進行測試

部署這兩項服務後，即可驗證 ADK 代理程式是否能順利與 GPU 加速的 Gemma 後端通訊，並回應使用者查詢。

1. 測試健康狀態端點：

```
curl $AGENT_URL/health
```

您應該會看到：

```
{ "status": "healthy", "service": "production-adk-agent" }
```

2. 在新瀏覽器分頁中輸入 `production-adk-agent` 的網址，即可與代理程式互動。您應該會看到 ADK 網頁介面。

3. 使用下列範例對話測試代理程式：

- 「小熊貓在野外通常吃什麼？」
- 「Can you tell me an interesting fact about snow leopards?」(可以告訴我關於雪豹的有趣知識嗎?)
- 「為什麼箭毒蛙的顏色這麼鮮豔？」
- 「動物園裡的新生小袋鼠在哪裡？」

👁️ 觀察重點：

- 代理程式會使用您部署的 Gemma 模型回覆。您可以觀察已部署 Gemma 服務的記錄，確認這項資訊。我們會在下一節中執行這項操作
- 回覆是由 GPU 加速後端生成
- 網頁介面提供簡潔的即時通訊體驗

 Agent Development Kit

producti... ▾

Trace

Events

Stat

Invocations

SESSION ID 827d4c8c-2639-4502-8f3a-060fb826acd4

Token Streaming

+

New Session

What's your name?





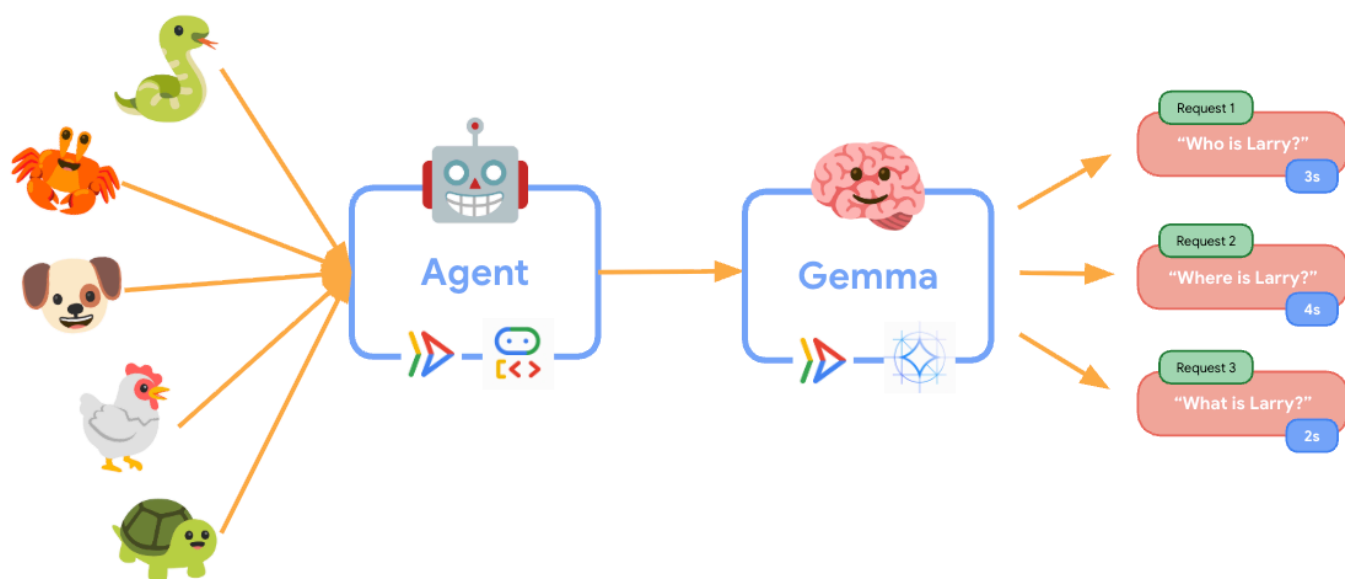
Type a Message...







11. 導入及執行彈性測試



為瞭解正式環境部署作業如何處理實際流量，我們將實作彈性測試，測試系統適應較高「模擬」正式環境工作負載的能力。

1. 開啟並實作彈性測試指令碼：

```
cloudshell edit elasticity_test.py
```

將 TODO 註解替換為：

```

import random
import uuid
from locust import HttpUser, task, between

class ProductionAgentUser(HttpUser):
    """Elasticity test user for the Production ADK Agent."""

    wait_time = between(1, 3) # Faster requests to trigger scaling

    def on_start(self):
        """Set up user session when starting."""
        self.user_id = f"user_{uuid.uuid4()}"
        self.session_id = f"session_{uuid.uuid4()}"

        # Create session for the Gemma agent using proper ADK API format
        session_data = {"state": {"user_type": "elasticity_test_user"}}

        self.client.post(
            f"/apps/production_agent/users/{self.user_id}/sessions/{self.session_id}",
            headers={"Content-Type": "application/json"},
            json=session_data,
        )

    @task(4)
    def test_conversations(self):
        """Test conversational capabilities - high frequency."""
        topics = [
            "What do red pandas typically eat in the wild?",
            "Can you tell me an interesting fact about snow leopards?",
            "Why are poison dart frogs so brightly colored?",
            "Where can I find the new baby kangaroo in the zoo?",
            "What is the name of your oldest gorilla?",
            "What time is the penguin feeding today?"
        ]

        # Use proper ADK API format for sending messages
        message_data = {
            "app_name": "production_agent",
            "user_id": self.user_id,
            "session_id": self.session_id,
            "new_message": {
                "role": "user",
                "parts": [{
                    "text": random.choice(topics)
                }]
            }
        }

        self.client.post(
            "/run",
            headers={"Content-Type": "application/json"},
            json=message_data,

```

```
)

@task(1)
def health_check(self):
    """Test the health endpoint."""
    self.client.get("/health")
```

🔧 這項功能的作用：

- **建立工作階段：**使用適當的 ADK API 格式，透過 POST 傳送至 `/apps/production_agent/users/{user_id}/sessions/{session_id}`。建立 `session_id` 和 `user_id` 後，即可向代理程式提出要求。
- **訊息格式：**遵循 ADK 規格，使用 `app_name`、`user_id`、`session_id` 和結構化 `new_message` 物件
- **對話端點：**使用 `/run` 端點一次收集所有事件 (建議用於負載測試)
- **符合現實的負載：**建立對話負載，縮短等待時間

📖 如要進一步瞭解 ADK API 端點和測試模式，請參閱 [ADK 測試指南](#)。

2. 安裝依附元件：

```
uv sync
```

3. Locust 是以 Python 為基礎的開放原始碼負載測試工具，專為網頁應用程式和其他系統的效能和負載測試而設計。主要特色是使用標準 Python 程式碼定義測試情境和使用者行為，相較於依賴圖形使用者介面或特定領域語言的工具，更具彈性和表現力。我們會使用 Locust 模擬服務的使用者流量。執行測試。

```
uv run locust -f elasticity_test.py \
    -H $AGENT_URL \
    --headless \
    -t 60s \
    -u 20 \
    -r 5
```

嘗試變更測試中的參數，並觀察輸出內容。🎨 **彈性測試參數：**

- **時間長度：**60 秒
- **使用者：**20 位並行使用者
- **產生率：**每秒 5 位使用者
- **目標：**觸發這兩項服務的自動調度資源

12. 觀察自動調度資源行為

執行彈性測試時，您會看到 Cloud Run 支援更高工作負載的實際情況。您可以在這裡瞭解將 ADK 代理程式與 GPU 後端分開的主要架構優勢。

在彈性測試期間，請在控制台中監控這兩項 Cloud Run 服務如何處理流量。

1. 在 **Cloud 控制台** 中，依序前往：

- Cloud Run → production-adk-agent → 指標
- Cloud Run → ollama-gemma3-270m-gpu → 「指標」

觀察重點：

ADK 代理服務：

- 流量增加時，應維持 1 個執行個體
- 流量高峰期間 CPU 和記憶體用量暴增
- 有效處理工作階段管理和要求轉送

Gemma 後端服務 (瓶頸)：

- 流量增加時，應維持 1 個執行個體
- 負載增加時，GPU 使用率大幅提升
- 由於模型推論需要大量 GPU 資源，這項服務會成為瓶頸
- 由於 GPU 加速，模型推論時間維持一致

重要洞察：

- 由於我們將執行個體數量上限設為 1，因此這兩項服務會保持一致，不會擴充
 - 這兩項服務會根據各自的負載特性獨立調度資源
 - GPU 有助於在不同負載條件下維持效能
-

13. 結論

恭喜！您已成功部署可供正式環境使用的 ADK 代理，並搭配 GPU 加速的 Gemma 後端，同時測試模擬的正式環境工作負載。

✅ 完成的目標

- ✅ 在 Cloud Run 上部署 GPU 加速的 Gemma 模型後端
- ✅ 建立並部署與 Gemma 後端整合的 ADK 代理
- ✅ 使用 ADK 網頁介面測試代理
- ✅ 觀察到兩個協調式 Cloud Run 服務的自動調度資源行為

💡 本實驗室的重點洞察

1. 🎮 **GPU 加速**：NVIDIA L4 GPU 可大幅提升模型推論效能
2. 🔗 **服務協調**：兩項 Cloud Run 服務可順暢地協同運作
3. 📈 **獨立調度資源**：每項服務都會根據個別的負載特性調度資源
4. 🚀 **適用於正式環境**：架構可有效處理實際流量模式

🔄 後續步驟

- 測試不同的負載模式，並觀察縮放行為
- 嘗試不同大小的 Gemma 模型 (視情況調整記憶體和 GPU)
- 為正式環境部署作業實作監控和快訊功能
- 探索多區域部署，確保全球可用性

🧹 清理

為避免產生費用，請在完成後刪除資源：

```
gcloud run services delete production-adk-agent --region=europe-west4
gcloud run services delete ollama-gemma3-270m-gpu --region=europe-west4
```

📖 資源

- [入門存放區](#)
- [完整解決方案](#)
- [Google ADK 說明文件](#)
- [ADK 測試指南](#) - ADK API 端點和測試模式的完整參考資料
- [在 Cloud Run 上進行負載測試](#)
- [Agent Development Kit \(ADK\) 說明文件](#)
- [Cloud Run GPU 說明文件](#)
- [Ollama 模型程式庫](#)
- [Google Cloud Trace 說明文件](#)
- [Cloud Run 安全性最佳做法](#)

- [UV Python 套件管理工具](#)
 - [Locust 負載測試架構](#)
-